Compiler Construction

Resolution - Normalization

Goals

Resolution

removes ambiguity and implicit behavior.

- Operators are bound to a single operation (integer addition, floatting-point addition ...)
- Casts are made explicit

Normalization

rewrites expressions into a uniform, compiler-friendly form.

- Return values must appear
- Some values are computed statically

Goals - Concretely

```
let x = 1 in
print (
  if x then
    type_of (x+"coucou") →
  else
    1.5
)
```

```
let x = 1 in
(
  print (
     if (x; true)
     then
       (concat(string_of_int(x); "coucou");
       "STRING")
     else
       string_of_float(1.5)
  );
  false
)
```

Overloading

Overloading: Homonyms

Several entities bearing the same name, but statically distinguishable, e.g., by their arity, type etc.

Aliasing: Synonyms

One entity bearing several names.

```
// foo is overloaded.
int foo(int);
int foo(float);

// x and y are aliases.
int x;
int& y = x;
```

Operator Overloading

Overloading simplifies the user's life (Since FORTRAN).

No Overloading in OCaml:

```
# 1.0 + 2.0;;
(* Characters 0-3:
  1.0 + 2.0;;
  ^^^
This expression has type float
but is here used with type int *)
```

Overloading in C:

```
int   a = 1   + 2;;
float b = 1.0 + 2.0;;
```

Function Overloading

Usually based on the arguments

Ada

```
I :  INTEGER;
X :  REAL;
...
PUT("results are: ");
PUT(I);
PUT(X);
```

Overloading is Syntactic Sugar

Overloaded

```
void foo(int);
void foo(char);
void foo(const char*);
void foo(std::string);

int main ()
{
  foo(0);
  foo('0');
  foo("0");
  foo(std::string("0"));
}
```

Overloading is Syntactic Sugar

Overloaded

```
void foo(int);
void foo(char);
void foo(const char*);
void foo(std::string);

int main ()
{
  foo(0);
  foo('0');
  foo("0");
  foo(std::string("0"));
}
```

De-overloaded

```
void foo_int(int);
void foo_char(char);
void foo_char_p(const char*);
void foo_std_string(std::string);

int main ()
{
  foo_int(0);
  foo_char('0');
  foo_char_p("0");
  foo_std_string(std::string("0"));
}
```

Overloading is Syntactic Sugar

Usually solved by renaming/mangling.

g++-2.95, como

```
f__Fi  -> int f(int);
f__FPc -> int f(char*);
```

g++-3.2, icc

```
_Z1fi  -> int f(int);
_Z1fPc -> int f(char*);
```

Overloading in ILP

Ordering <, <=, >, and >=
overloaded for pairs of integers, or strings.

Identity = and <>
overloaded for (type coherent) pairs of integers, strings, arrays or records.

Concatenation +
overloaded for everything

Implicit in ILP

ILP source programs contain many **implicit features**:

- Operator overloading
- Implicit casts between basic types
- Polymorphic primitives (`print`, `type_of`, +, ...)
- Missing return values
- Missing `else` branches

These must be eliminated before IR generation.

Resolution Phase

Resolution makes everything **explicit**:

- Every operator is renamed to a unique, typed operator
- Every implicit cast becomes an explicit function call
- Control-flow always produces a value
- Side effects are made explicit using sequences

Resolver: Global View

The resolver is a **TAST visitor**:

- Traverses the tree top-down
- Rewrites expressions when needed

WARNING: We must preserve the program semantics

- evaluation order must not change
- side-effects must be kept

```
Resolver implements ITASTvisitor<ITASTexpression>
```

Operator Resolution

Operators are resolved using:

- The operator environment
- Operand static types

Example:

- + becomes:
  - `ILP_Plus_INT`
  - `ILP_Plus_FLOAT`
  - `concat` (for strings)

Resolving +

```
if (lt == STRING || rt == STRING)
  concat(string(l), string(r))
else if numeric
  ILP_Plus_INT or ILP_Plus_FLOAT
```

The overloaded operator disappears.

Implicit Casts

ILP allows implicit casts:

- `int →float`
- `int →string`
- `bool →string`

Resolution inserts explicit conversion calls:

- `string_of_int`
- `float_of_int`
- conditional expressions for booleans

Cast Insertion - Strings

Wraps a typed expression into an invocation that converts it to target type.

```
private ITASTexpression castIfNeeded(ITASTexpression expr, Type to) {
  Type from = expr.getType();
  if (from == to) return expr; //no cast is required
  String fnName = null;

  if (to == Type.STRING) {
    if (from == Type.INT) fnName = "string_of_int";
    else if (from == Type.FLOAT) fnName = "string_of_float";
    else if (from == Type.BOOL)
    return new TASTalternative(expr,trueString,falseString,Type.STRING);
  }

  ITASTvariable fnVar = new TASTvariable(fnName, Type.FUNCTION);
  return new TASTinvocation(fnVar, new ITASTexpression[]{expr}, to);
}
```

Cast Insertion - Strings

Why it works?

- We suppose the existence at runtime of casting functions: `string_of_int`, `int_of_string` ...
- Special case for booleans
  - rewritten to an alternative `expr ? "true" : "false"`
  - No runtime function call needed

Same thing holds for casting to floats and ints.

Cast Insertion - Booleans

Wraps a typed expression into an sequence that returns `true`.

```
if (from == to) return expr; // no cast required

...

if (to == Type.BOOL) {
  // wrap the expression into a boolean-producing sequence
  ITASTexpression[] arr = new ITASTexpression[2];
  arr[0] = expr;                          // preserve side-effects
  arr[1] = new TASTboolean("true", Type.BOOL);
  return new TASTsequence(arr, Type.BOOL);
}

...
```

Cast Insertion - Booleans

Why it works?

- Only `false` has the boolean value of falseness (encoded as `0`)
- Any other type (int, float, string, etc.) is always considered `true` when cast
- We can guarantee this statically without inspecting the runtime value

Preserving side effects:

- Simply replacing `expr` with `true` would skip any side-effects in `expr`
- Wrapping `expr` in a sequence ensures it is evaluated first

Equality and Comparisons

Equality operators:

- Numeric equality (int / float)
- Boolean equality
- String equality

Mixed or incompatible types are resolved statically.

Static Resolution

Given two expression e_1 of type τ_1 and e_2 of type τ_2

Resolve *statically*:

- $e_1 = e_2$ when $\tau_1 = \mathit{INT}$ and $\tau_2 = \mathit{STRING}$
- $e_1 \neq e_2$ when $\tau_1 = \mathit{INT}$ and $\tau_2 = \mathit{STRING}$

Static Resolution

Given two expression e_1 of type τ_1 and e_2 of type τ_2

Resolve *statically*:

- $e_1 = e_2$ when $\tau_1 = \mathit{INT}$ and $\tau_2 = \mathit{STRING} \rightarrow$ can be replaced by `false`
- $e_1 \neq e_2$ when $\tau_1 = \mathit{INT}$ and $\tau_2 = \mathit{STRING} \rightarrow$ can be replaced by `true`

Static Resolution

Given two expression e_1 of type τ_1 and e_2 of type τ_2

Resolve *statically*:

- $e_1 = e_2$ when $\tau_1 = \mathit{INT}$ and $\tau_2 = \mathit{STRING} \rightarrow$ can be replaced by `false`
- $e_1 \neq e_2$ when $\tau_1 = \mathit{INT}$ and $\tau_2 = \mathit{STRING} \rightarrow$ can be replaced by `true`

WRONG!!

Counter-example: `(x=1)=="coucou"`

$e_1 \neq e_2$ when $\tau_1 = \mathit{INT}$ and $\tau_2 = \mathit{STRING} \rightarrow$ can be replaced by $(e_1; e_2;\texttt{true})$

Control Flow Normalization

In ILP, `if-then-else` are expressions.

- Condition must be explicitly converted to a boolean
- Both branches must have the same type
- A missing `else` must be synthesized

`if cond then expr` $\rightarrow$ `if bool(cond) then expr else default`

Default:

- int $\rightarrow$ 0
- float $\rightarrow$ 0.0
- bool $\rightarrow$ false
- string $\rightarrow$ ""

About `to_string` & `print`

`to_string` is an implicit cast. Dispatch according to the argument type:

- `string_of_int`
- `string_of_float`

We could do the same thing for `print` and dispatch it to `print_int` ... But this requires adding them in our runtime.

Instead, we exploit the fact that `print(x)` is equivalent to `print(to_string x)`

Primitive Functions

Static typing of ILP enables static resolution of `type_of`!

example: `type_of(2+2)` $\rightarrow$ "INT"

Primitive Functions

Static typing of ILP enables static resolution of `type_of`!

example: `type_of(2+2)` $\rightarrow$ "INT"

WARNING:
`type_of(print("coucou");2)` $\neq$ "INT"
Side-effects must be kept

Other identifiers are resolved statically:

- `pi` $\longrightarrow$ constant float value

Avoids unnecessary runtime lookups.

End Result

After resolution:

- No overloaded operators
- No implicit casts
- Explicit return values
- Fully typed AST

Ready for IR generation.

ILP2 Resolver: Design

The ILP2 resolver:

- Inherits from the ILP1 resolver
- Reuses all operator and primitive resolution
- Adds support for new language constructs

- assignment
- loops
- function definition

ILP2 Resolver: Design

The ILP2 resolver:

- Inherits from the ILP1 resolver
- Reuses all operator and primitive resolution
- Adds support for new language constructs

- assignment — easy
- loops — easy
- function definition — interesting

Function Definitions

In ILP2, functions are:

- Top-level and globally visible
- Callable from any function body
- Potentially mutually recursive

Resolution must therefore handle **forward references**.

Two-Phase Resolution - 1

```
function f(x:int):int = g(x)
function g(y:int):int = y + 1
```

When resolving the body of `f`:

- `g` has not been visited yet
- But `g` is a valid function

A single-pass resolver would incorrectly reject this program.

Two-Phase Resolution - 2

Resolution is split into two distinct concerns:

1. **Register function signatures** (names, parameters, return types)
   Bodies are left untouched.
2. **Resolve function bodies**
   with all functions already known.

This allows forward references and mutual recursion while keeping resolution simple and deterministic.

Function Invocation: User-Defined *vs* Primitives

If the function name matches a user definition:

- Arguments are resolved recursively
- Invocation is rebuilt as-is

No overloading or dynamic dispatch.

If the function is not user-defined:

- Delegated to the ILP1 resolver
- Handles `print`, `type_of`, conversions, etc.

Ensures backward compatibility.

Conclusion

Resolution is desugaring + typing + normalization.

It bridges the gap between:

User-friendly language

and

Compiler-friendly representation

Everything remains:

- Typed
- Explicit
- Deterministic